Διαχείριση Σύνθετων Δεδομένων

Εργασία 3: Indexing and Similarity Search for Dense Multidimensional Vectors

Παυλίδης Ελευθέριος – 5332

Σωτήριος Ζώτος - 5227

Μέρος 1: Φόρτωση Δεδομένων και Υπολογισμός Pivots

Υλοποιήσαμε τη συνάρτηση find_refs() η οποία επιλέγει τα pivots με την εξής διαδικασία:

1. Ορίζεται ως seed το πρώτο αντικείμενο του D.
2. Το πρώτο pivot είναι το πιο απομακρυσμένο αντικείμενο από το seed.
3. Κάθε επόμενο pivot επιλέγεται ως το αντικείμενο με το μέγιστο άθροισμα αποστάσεων από όλα τα ήδη επιλεγμένα pivots.

Η συνάρτηση επιστρέφει ένα 2D array distances[i][j] με την απόσταση κάθε αντικειμένου i από κάθε pivot j.

Output (numpivots = 5):

Παράμετρος	Τιμή
Αριθμός pivots (numpivots)	5
Pivots που επιλέχθηκαν	[1109, 6878, 5514, 5026, 5338]

Μέρος 2: Μέθοδος iDistance

Υλοποιήσαμε τη συνάρτηση compute_idistance_array() η οποία:

1. Για κάθε αντικείμενο o βρίσκει το κοντινότερο pivot npiv(o).
2. Υπολογίζει το maxd(p): μέγιστη απόσταση προς οποιοδήποτε o για το οποίο το p είναι το κοντινότερο pivot.
3. Υπολογίζει το maxd = max{ maxd(p) } για όλα τα pivots.
4. Υπολογίζει την iDistance τιμή κάθε αντικειμένου: iDist(o) = i * maxd + dist(o, pi), όπου pi το κοντινότερο pivot.
5. Ταξινομεί το array κατά iDistance τιμή.

Output (numpivots = 5):

Παράμετρος	Τιμή
maxd	1.7476
maxd_per_pivot[0]	1.7014
maxd_per_pivot[1]	1.7102
maxd_per_pivot[2]	1.7476
maxd_per_pivot[3]	1.6536
maxd_per_pivot[4]	1.6399

Πρώτες 10 εγγραφές του iDistance array:

```
(0.0,          1109, 0)
(0.5209740876, 7409, 0)
(0.5490364287, 8441, 0)
(0.5580519689, 6847, 0)
(0.5724596056, 3621, 0)
(0.6028938547, 2912, 0)
(0.6199056380, 54,   0)
(0.6227631974, 9287, 0)
(0.6355761166, 5943, 0)
(0.6357216372, 2021, 0)
```

Μέρος 3: Range Similarity Queries

Υλοποιήσαμε τρεις μέθοδους για range queries (numpivots = 10, ε = 0.2):

a) Naive: linear scan όλων των αντικειμένων — πάντα 10.000 distance computations.
b) Pivot-based: χρησιμοποιεί την ανισότητα τριγώνου για pruning. Υπολογίζει εκ των προτέρων τις αποστάσεις query→pivots (len(refs) computations) και αποκλείει αντικείμενα για τα οποία |dist(pi,o) - dist(pi,q)| > ε.
c) iDistance: binary search στο ταξινομημένο array για εύρεση candidates ανά partition, χωρίς scan όλων των αντικειμένων.

Output:

Μέθοδος	Avg Distance Comps	Total Time (s)
Naive	10000.0	5.4705
Pivot-based	16.71	1.0249
iDistance	2074.24	1.4839

Μέρος 4: kNN Similarity Queries

Υλοποιήσαμε τρεις μέθοδους για kNN queries (numpivots = 10, k = 5):

a) Naive: χρησιμοποιεί max-heap μεγέθους k για να κρατά τα k κοντινότερα αντικείμενα.
b) Pivot-based: χρησιμοποιεί δυναμικό r (κορυφή του heap) για pruning μέσω της ανισότητας τριγώνου.
c) iDistance: σαρώνει πρώτα το partition του κοντινότερου pivot στο query (χωρίς pruning), μετά τα υπόλοιπα partitions με pruning βάσει της τρέχουσας kNN-οστής απόστασης.

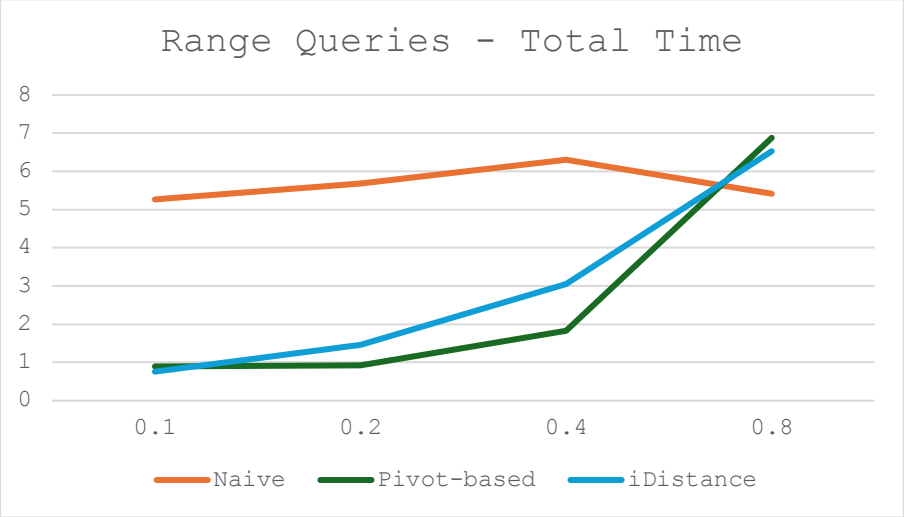
Output:

Μέθοδος	Avg Distance Comps	Total Time (s)
Naive	10000.0	5.7619
Pivot-based	3156.755	3.7964
iDistance	6128.03	4.2953

Μέρος 5: Αξιολόγηση

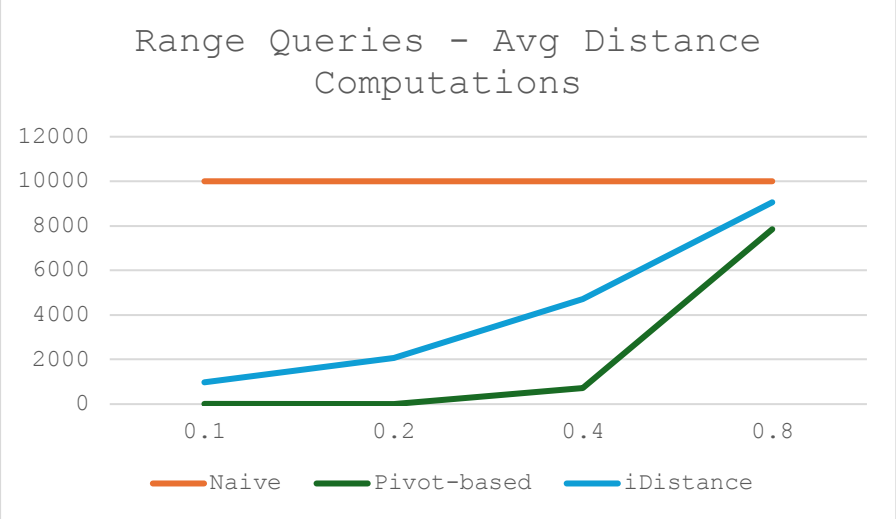
5.1 Range Queries — Συνολικός Χρόνος (numpivots = 10)

epsilon	Naive (s)	Pivot-based (s)	iDistance (s)
0.1	5.2666	0.8958	0.7597
0.2	5.6815	0.9218	1.4579
0.4	6.3041	1.8258	3.0601
0.8	5.4079	6.8772	6.5280



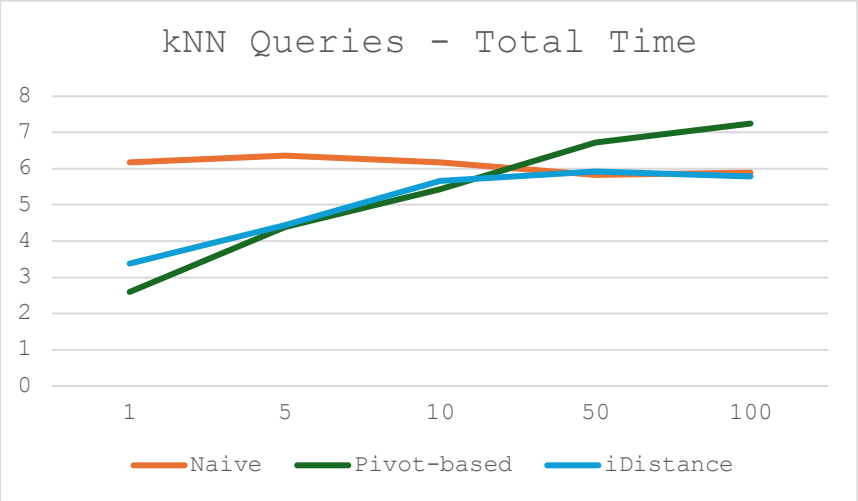
5.2 Range Queries — Μέσος Αριθμός Distance Computations

epsilon	Naive	Pivot-based	iDistance
0.1	10000.00	10.03	978.07
0.2	10000.00	16.71	2074.24
0.4	10000.00	726.98	4703.69
0.8	10000.00	7847.74	9056.80



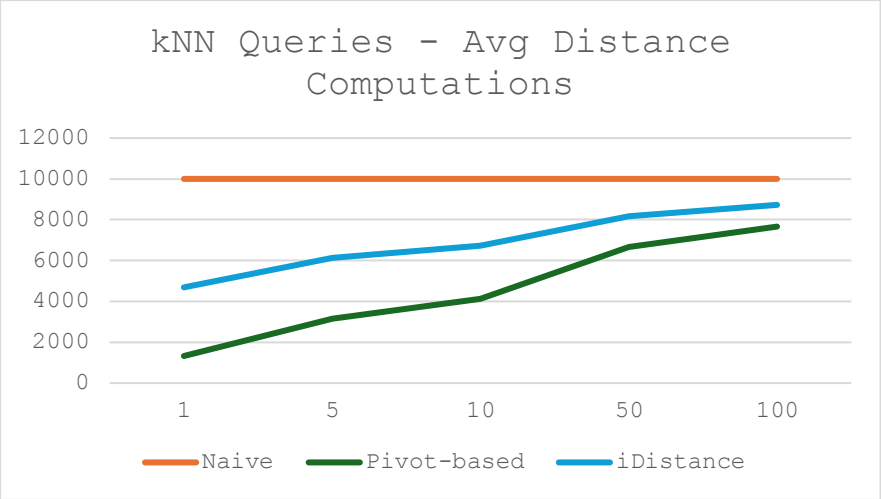
5.3 kNN Queries — Συνολικός Χρόνος (numpivots = 10)

k	Naive (s)	Pivot-based (s)	iDistance (s)
1	6.1728	2.5966	3.3798
5	6.3608	4.3897	4.4479
10	6.1724	5.4346	5.6538
50	5.8283	6.7181	5.9218
100	5.8997	7.2468	5.7858



5.4 kNN Queries — Μέσος Αριθμός Distance Computations

k	Naive	Pivot-based	iDistance
1	10000.00	1325.08	4688.85
5	10000.00	3156.76	6128.03
10	10000.00	4139.14	6737.77
50	10000.00	6653.55	8168.35
100	10000.00	7663.78	8729.35



5.5 Παρατηρήσεις

Range Queries:

Για μικρές τιμές του ε (0.1, 0.2), η pivot-based μέθοδος είναι αποδοτική με μόλις 10-17 distance computations ανά query, καθώς η πλειοψηφία των αντικειμένων αποκλείεται από τα pivots. Το iDistance έχει σημαντικά περισσότερα computations (978-2074) αλλά παραμένει καλύτερο από την naive.

Καθώς το ε αυξάνεται (0.4, 0.8), και οι δύο μέθοδοι χάνουν την αποδοτικότητά τους: για ε=0.8 το pivot-based φτάνει τα 7848 computations και το iDistance τα 9057, διότι ο μεγάλος radius καλύπτει σχεδόν όλο το χώρο και το pruning γίνεται αναποτελεσματικό. Στο ίδιο σημείο, η naive γίνεται ανταγωνιστική σε χρόνο.

kNN Queries:

Για μικρά k (1, 5), η pivot-based είναι η πιο αποδοτική μέθοδος με λίγα computations, καθώς το δυναμικό r (kNN-οστή απόσταση) γρήγορα γίνεται μικρό επιτρέποντας επιθετικό pruning. Το iDistance έχει περισσότερα computations λόγω της απαίτησης να σαρωθεί πλήρως το πρώτο partition.

Για μεγάλα k (50, 100), η pivot-based επιβαρύνεται περισσότερο από τη naive σε χρόνο, διότι το r παραμένει μεγάλο και δεν επιτρέπει αποτελεσματικό pruning. Το iDistance αποδίδει καλύτερα σε χρόνο από την pivot-based για μεγάλα k, λόγω της δομημένης πρόσβασης στα partitions.

Γενικά συμπεράσματα:

Η pivot-based μέθοδος είναι καλύτερη για μικρά ε και k. Το iDistance αποδίδει καλύτερα όταν τα queries έχουν μεγαλύτερο radius ή k, ενώ για πολύ μεγάλα ε/k και οι τρεις μέθοδοι συγκλίνουν σε απόδοση.